CS683 2026 Assignment 2 Report

24B1020, 24B1046, 24B1078, 24B1082

Contents

1 Methodology **2**

2 Task 1A: IP-stride prefetcher **2**

3 Task 1B: Stride Pattern Prefetcher (SPP) **3**
3.1 Design . . . 3
3.2 Accuracy-based throttling . . . 3
3.3 Storage overhead . . . 4
3.4 Results . . . 4
3.5 Discussion . . . 6

4 Task 2A: Instruction-conscious L2 replacement (I-RRIP) **6**
4.1 Design and rationale . . . 6
4.2 Results . . . 7
4.3 Discussion . . . 8

5 Task 2B: Dynamic I-RRIP **8**
5.1 Design . . . 8
5.2 Sensitivity study . . . 9
5.3 Results . . . 9
5.4 Discussion . . . 10

6 Task 3: SPP with prefetch-aware Dynamic I-RRIP **11**
6.1 Design and rationale . . . 11
6.2 Results . . . 11
6.3 Discussion . . . 15

1 Methodology

We use the ChampSim simulator from the pa2 repository with a single core. Cache configuration was fixed in the repository, and we are to change only the prefetching policy for L1D cache and the replacement policy of the L2C cache. The other caches have no prefetching and LRU replacement policy. Each binary was built with `./build_champsim.sh <L1D prefetcher> <L2C replacement> <name>` and run for 25M warmup instructions followed by 25M simulated instructions.

The baseline for the speedup in this report is no L1D prefetcher with LRU replacement at the L2C.

We are running the simulator over 3 different traces, and computing the metrics required by the question - either these are directly in the statistics of ChampSim or can be easily computed from that.

2 Task 1A: IP-stride prefetcher

The IP-stride prefetcher (`prefetcher/ip_stride.l1d_pref`) keeps a table of 24 IP trackers, one per recently seen instruction pointer (IP). Each tracker stores the IP, the last cache-line address that IP accessed, the last stride it observed, and an LRU position.

On an L1D load the prefetcher looks the IP up in the table. Only the last 8 bits of the address is stored. If the IP is not being tracked, the least recently used tracker is evicted and reassigned to it; the current cache-line address is recorded and the last stride is set to 0. No prefetch is issued. If the IP is already tracked, the stride between the current cache-line address and the tracker's last address is computed. When this stride equals the tracker's previous stride (the equality makes sure that we see the ip more than once before prefetching), the prefetcher issues prefetches for the next 3 lines (the fixed degree), moving by the same stride each time: $\text{addr} + \text{stride}$, $\text{addr} + 2 \cdot \text{stride}$ and $\text{addr} + 3 \cdot \text{stride}$. The tracker's last address and last stride are then updated and it becomes the most recently used entry.

Addresses are shifted right by `LOG2_BLOCK_SIZE` before strides are computed, since data moves through the hierarchy in whole cache lines, so strides are measured in lines rather than bytes. A stride of 0 is ignored, because it means the same IP accessed the same cache line again (for example two loads a few bytes apart within one line), so there is nothing new to predict.

Each prefetch is filled into L1 only if the L1D MSHR is less than half full; otherwise it is filled only into L2. The MSHR tracks outstanding misses, and prefetches are speculative, so when it is getting full the remaining entries are left for demand misses. The prefetched line still moves closer to the core by being placed in L2.

3 Task 1B: Stride Pattern Prefetcher (SPP)

3.1 Design

SPP (`prefetcher/spp.l1d_pref`) extends the IP-stride tracker table so that each IP can be in one of two modes. It starts the uniform (constant-stride) mode and goes to non-uniform (repeating multi-stride) prediction when the uniform pattern breaks.

Every newly allocated IP starts in uniform mode, where it behaves like the IP-stride prefetcher: when the current stride equals the last stride, it prefetches along that stride. A per-tracker mismatch counter (`num_fails`) is incremented on every stride mismatch and decremented on every match (the minimum value is 0). When it reaches `TRIGGER` $= 3$, the IP is considered non-uniform, the counter is reset, and the IP switches to non-uniform mode.

In non-uniform mode the tracker first records the next `NO_OF_REPEATS` $\times$ `MAX_PATTERN_LENGTH` $= 3 \times 12 = 36$ strides in a history buffer. Once the buffer is full, every candidate pattern length $p = 1, 2, \ldots, 12$ is tested in increasing order. A length p is accepted if history$[k]$ = history$[k+p]$ holds for all k, with fewer than 3 mismatches, so the shortest repeating period is chosen and a little noise is tolerated. The accepted pattern is copied from the second repetition in the buffer, and the prediction index is aligned to the current position in that period. If no length is accepted, the buffer is cleared and learning starts again.

The stored pattern together with the current position in it acts as the IP's signature of recently observed strides, and this pair maps directly to the next expected stride. When the observed stride matches the predicted one, the prefetcher walks ahead along the pattern, adding up the strides, and issues one prefetch per step up to the current degree. For example, for the pattern $(+1, +2, +2, +2, +3)$ at position 0 with degree 3, it prefetches $A + 2$, $A + 4$ and $A + 6$. The position then advances by one (modulo the pattern length). On a misprediction the mismatch counter is incremented; after three mispredictions the pattern is discarded, the history is cleared, and the IP relearns from scratch.

3.2 Accuracy-based throttling

The prefetch degree is recomputed on every L1D access from the L1D's built-in counters, as accuracy $= \frac{\texttt{pf_useful}}{\texttt{pf_lower_level}}$. We got this formula from how the prefetch accuracy was calculated in `main.cc` in Champsim. `pf_useful` counts prefetched lines that were later hit by a demand load, and `pf_lower_level` counts prefetches that actually missed in L1D and were sent to L2. ChampSim resets these counters at the end of warmup, so the accuracy is measured over the simulated region.

The degree is set to 6 above 90% accuracy, 5 for 80–90%, 4 for 75–80%, 3 for 60–75%, 2 for 20–60%, and 1 below 20%.

This is because, if our prefetcher is accurate, we can keep increasing the degree. When the accuracy goes down, we should no longer increase the degree of the

prefetcher. These values are small because we want to avoid cache pollution.

3.3 Storage overhead

Each of the 24 tracker entries holds the IP, the last cache-line address and last stride (as in IP-stride), a 36-stride history buffer used while learning, a 12-stride pattern buffer, and a few extra fields: the pattern length, the history index, the prediction index, the LRU position, the uniform and pattern-found flags, and the mismatch counter. Throttling only adds one more variable as we are using the remaining from Champsim itself.

3.4 Results

Table 1: Task 1B results. Speedup is relative to the no-prefetcher LRU baseline.

Trace	Prefetcher	IPC	Speedup	L1D load MPKI	Accuracy (%)	Coverage
Trace1	Baseline	0.232345	1.0000	10.678	nan	nan
Trace1	IP-stride	0.233452	1.0047	10.119	52.7133	0.0523
Trace1	SPP	0.233011	1.0029	10.3921	52.8476	0.0267
Trace2	Baseline	0.251158	1.0000	13.5379	nan	nan
Trace2	IP-stride	0.25126	1.0004	13.4378	38.1823	0.0074
Trace2	SPP	0.251268	1.0004	13.4266	46.1722	0.0082
Trace3	Baseline	0.382065	1.0000	3.40344	nan	nan
Trace3	IP-stride	0.382862	1.0021	3.31816	15.9918	0.250
Trace3	SPP	0.38223	1.0004	3.37792	20.0238	0.0074

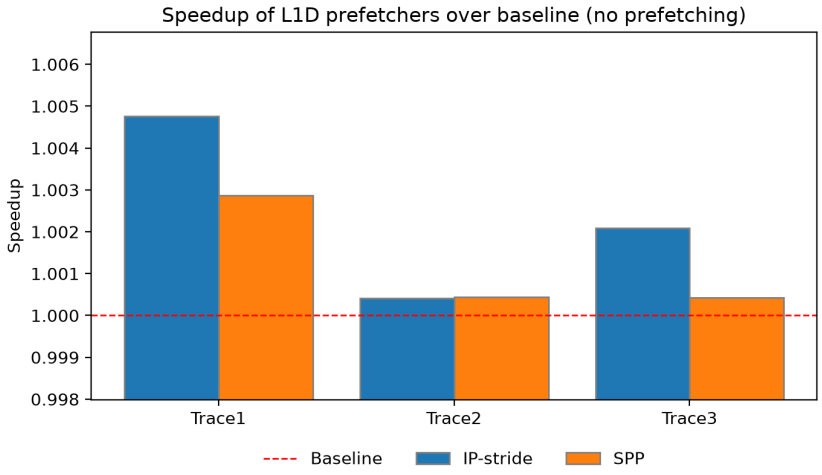


Figure 1: Task 1: Speedup of L1D prefetchers over the baseline (no prefetching).

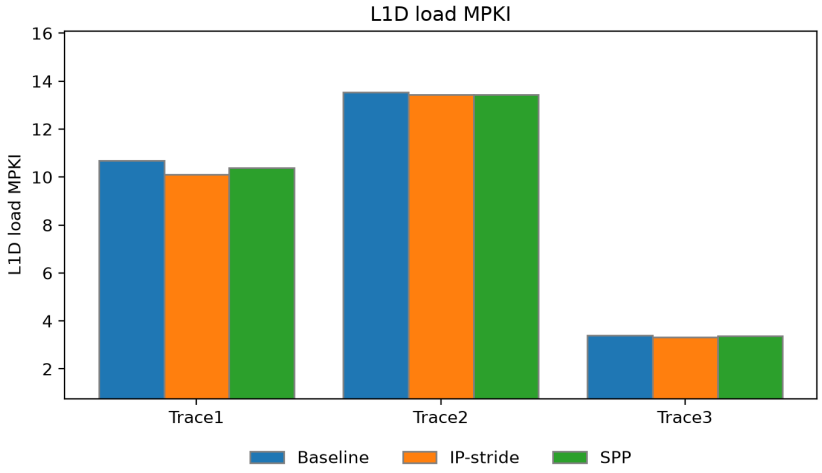


Figure 2: Task 1: L1D load MPKI.

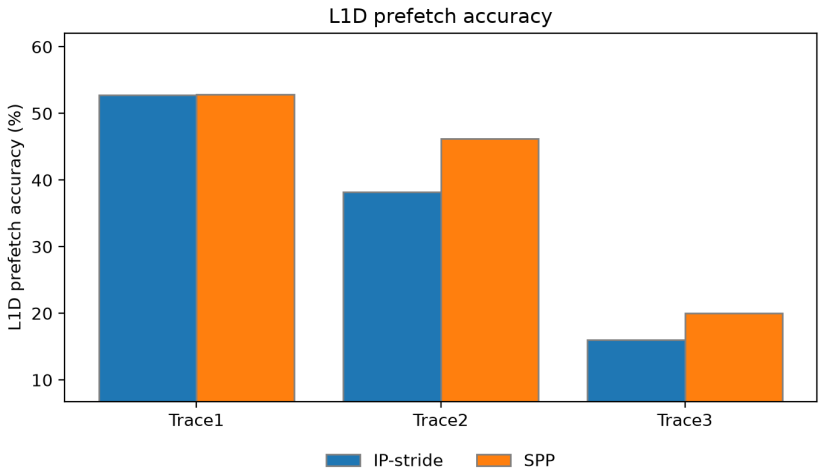


Figure 3: Task 1: L1D prefetch accuracy.

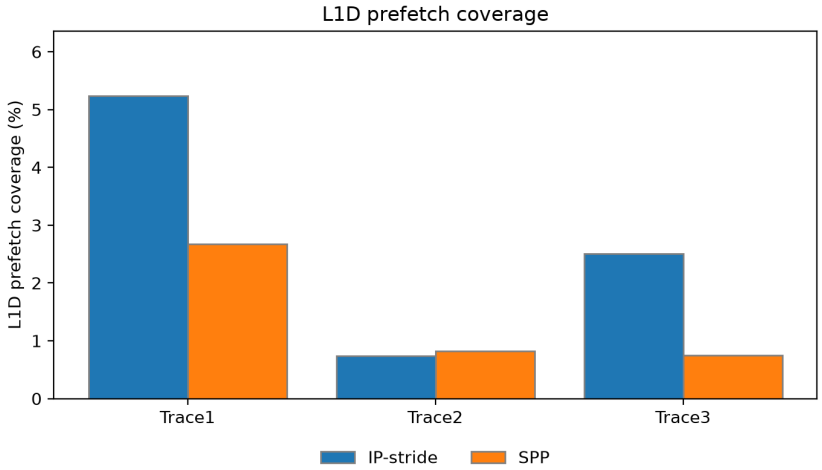


Figure 4: Tsk 1: L1D prefetch coverage.

3.5 Discussion

Both the IP stride prefetcher and the SPP prefetcher gave speedup compared to the baseline, and they also lowered the L1D load misses per 1000 instructions. Between IP stride and SPP, SPP has the better accuracy on all three traces, which means it is predicting correctly if it does. But the coverage is less (or very slightly more in trace 2) because the accuracy is in the range such that the degree of prefetch is 2, whereas the IP stride did 3. So SPP didn't do enough "miss removal", but it was better at predicting.

4 Task 2A: Instruction-conscious L2 replacement (I-RRIP)

4.1 Design and rationale

I-RRIP (`replacement/irrip.l2c_repl`) starts from RRIP with 2-bit re-reference prediction values (RRPV, 0 = near re-use, 3 = distant). We extended `update_replacement_state` with an `isInstruction` argument, taken from the packet's `instruction` field, so the policy knows whether each line holds an instruction or not.

For insertion, instruction lines are inserted with RRPV = 1 and all other lines with RRPV = 2 (the RRIP default). This gives an instruction line one extra round of ageing before it can become an eviction candidate, so it survives longer than a data line inserted at the same time. We did not insert instructions at RRPV = 0, because that would treat every instruction fill as certain to be re-used and let cold code that runs only once stay in the cache as long as genuinely hot lines.

We leave promotion unchanged. On a hit, both instruction and data lines go to RRPV = 0. A hit shows that the line is being re-used whatever it holds, and demoting re-used data lines would make hot data easier to evict than cold instructions, which works against reducing total misses. Instructions already get priority at insertion and eviction, so we felt there is no need to treat them differently here as well.

For eviction, among the lines at RRPV = 3 a data line is always chosen before an instruction line, and an instruction line is evicted only if no data line has reached RRPV = 3. If no line is at RRPV = 3, all lines are aged by one and the search repeats, as in RRIP. So, an instruction line has to age at least as far as every data line in the set before it is chosen.

4.2 Results

Table 2: Task 2A results. Speedup is relative to the LRU baseline (no prefetching).

Trace	L2C policy	IPC	Speedup	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
Trace1	LRU	0.232345	1.0000	17.6956	10.3377	7.35796
Trace1	RRIP	0.232231	0.9995	17.6387	10.5618	7.07688
Trace1	I-RRIP	0.232315	0.9999	17.5461	10.0303	7.51584
Trace2	LRU	0.251158	1.0000	27.3568	18.8533	8.50352
Trace2	RRIP	0.251806	1.0026	25.7458	17.7813	7.96452
Trace2	I-RRIP	0.255687	1.0180	23.0962	14.2311	8.86512
Trace3	LRU	0.382065	1.0000	3.95424	1.12408	2.83016
Trace3	RRIP	0.381739	0.9991	4.03948	1.19704	2.84244
Trace3	I-RRIP	0.381909	0.9995	3.95256	1.0764	2.87616

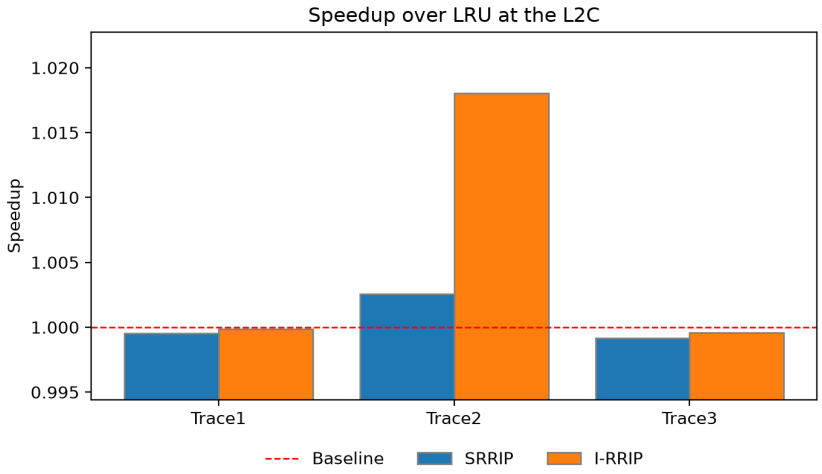


Figure 5: Task 2A: Speedup of RRIP and I-RRIP over the LRU baseline.

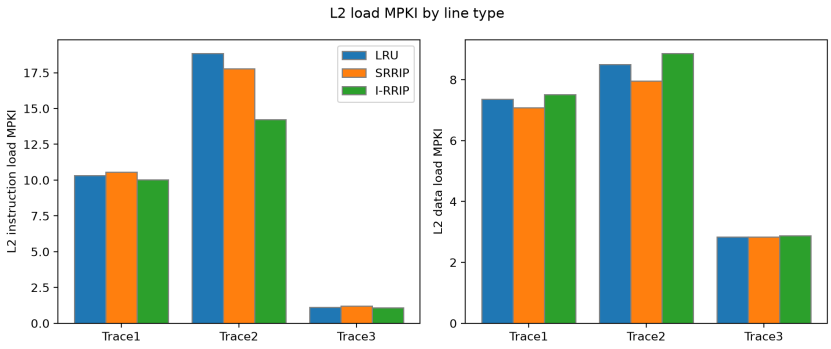


Figure 6: Task 2A: L2 instruction and L2 data load MPKI.

4.3 Discussion

In every trace, IRRIP outperformed RRIP, especially in Trace 2 which was instruction heavy, wherein it outperformed even LRU. In traces 1 and 3, which were not as instruction heavy, LRU $\geq$ IRRIP $\geq$ RRIP in terms of speedup. The L2C instruction MPKI shows a significant reduction, at the cost of increasing the data MPKI, because we are keeping more instructions in the cache (which makes it likely that instructions will get hit), but evicting more data (more writeback misses from L1 happen because of this).

5 Task 2B: Dynamic I-RRIP

5.1 Design

Dynamic I-RRIP (`replacement/dyn_irrip.l2c_repl`) uses set dueling to switch the follower sets between I-RRIP and plain RRIP.

A set i is a leader set if $i \bmod \lfloor 1/f \rfloor = 0$, where f is the leader fraction, so the leaders are spread evenly across the index range. Leader sets always run plain RRIP, similar to how there are leader sets in dynamic insertion policy. In DIP, there are 2 groups of leader sets, each running one of the policies. Since the problem statement only mentions the SDMs for the RRIP policy, we implemented that. We can get an idea of how the algorithm without caring about instructions performs, so that we can know whether caring about instructions is good or not after a point.

In the leader sets, and only for `LOAD` requests, we keep four counters: total accesses, total hits, instruction accesses and instruction hits. We want the ratio to indicate only the loads (demanded), so everything else is left out. Misses are taken as accesses minus hits. Then we compute the ratio

$$r = \frac{\text{instr. accesses} + \text{instr. misses}}{\text{total accesses} + \text{total misses}} = \frac{2 \cdot \text{instr. accesses} - \text{instr. hits}}{2 \cdot \text{total accesses} - \text{total hits}},$$

with $r = 0$ when the denominator is 0. If r is above a fixed threshold, the follower

sets use I-RRIP insertion and eviction implemented in 2a, otherwise they behave as RRIP. Promotion is the same in both modes.

As we want the policy to adapt to the load, and ratios of big numbers change very slowly, we half all the counters after the total access count of the leader sets reaches `L2C_SET`. This makes sure we don't lose all the progress, but the ratio keeps adapting dynamically.

5.2 Sensitivity study

We evaluated five thresholds for each of two leader fractions. This sweep was run on Trace1 only.

Table 3: Task 2B sensitivity study on Trace1. Speedup is relative to the LRU baseline.

Threshold	Leader fraction	IPC	Speedup	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
0.40	0.05	0.232271	0.9997	17.5401	10.0528	7.48724
0.45	0.05	0.232327	0.9999	17.5318	10.0655	7.46632
0.50	0.05	0.232342	0.99999	17.5212	10.0816	7.43964
0.55	0.05	0.232376	1.0001	17.4964	10.1698	7.32652
0.60	0.05	0.232304	0.9998	17.5262	10.3097	7.21648
0.40	0.10	0.232273	0.9997	17.5664	10.1021	7.46428
0.45	0.10	0.232314	0.9999	17.5518	10.1036	7.44812
0.50	0.10	0.232290	0.9998	17.5393	10.1283	7.41104
0.55	0.10	0.232376	1.0001	17.5242	10.1982	7.32592
0.60	0.10	0.232369	1.0001	17.5106	10.2955	7.21504

5.3 Results

Table 4: Task 2B results with threshold 0.40 and leader fraction 0.05 Speedup is relative to the LRU baseline.

Trace	L2C policy	IPC	Speedup	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
Trace1	RRIP	0.232231	0.9995	17.6387	10.5618	7.07688
Trace1	I-RRIP	0.232315	0.9999	17.5461	10.0303	7.51584
Trace1	Dynamic I-RRIP	0.232271	0.9996	17.5401	10.0528	7.48724
Trace2	RRIP	0.251806	1.0026	25.7458	17.7813	7.96452
Trace2	I-RRIP	0.255687	1.0180	23.0962	14.2311	8.86512
Trace2	Dynamic I-RRIP	0.255444	1.0171	23.2616	14.4482	8.81336
Trace3	RRIP	0.381739	0.9991	4.03948	1.19704	2.84244
Trace3	I-RRIP	0.381909	0.9995	3.95256	1.0764	2.87616
Trace3	Dynamic I-RRIP	0.381757	0.9992	4.0358	1.17076	2.86504

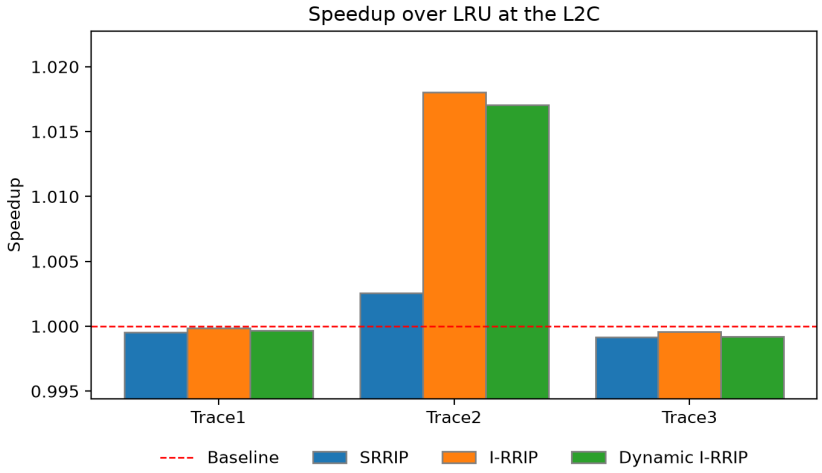


Figure 7: Task 2B: Speedup of RRIP, I-RRIP and dynamic I-RRIP over the LRU baseline.

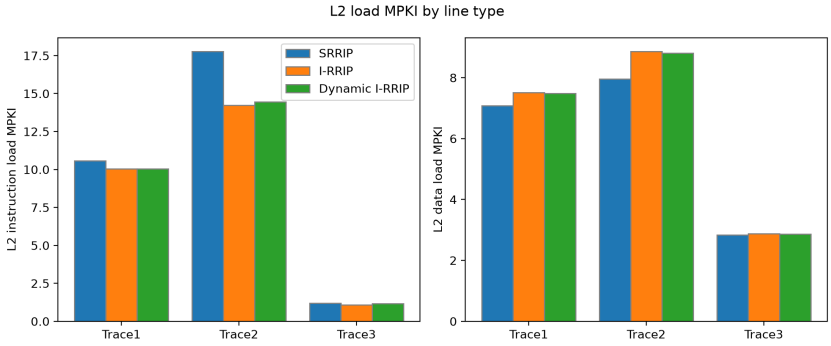


Figure 8: Task 2B: L2 instruction and L2 data load MPKI.

5.4 Discussion

This policy attempts to get the best of both worlds - instruction MPKI is low in IRRIP and data MPKI is low in RRIP. A higher threshold means the policy picks I-RRIP less often. With increase in threshold, the instruction MPKI increases and data MPKI decreases, and the total load MPKI first decreases and then increases. Changing the leader fraction did not have much of a difference between 0.05 and 0.1 because less number of sets was enough to gauge the performance of RRIP.

Although the best threshold on trace 1 was 0.55, we ran the simulator with 0.40 as the threshold (as we forgot to change it after running the sweep). It is more close to the IRRIP, but performs worse due to the leader sets being forced to follow RRIP even though IRRIP works much better in trace 2. The advantage at this threshold is not felt, but dynamic I-RRIP did improve the IPC in trace 1 when the threshold was 0.55 so changing the threshold could fix the problem.

6 Task 3: SPP with prefetch-aware Dynamic I-RRIP

6.1 Design and rationale

Task 3 extends `replacement/dyn_irrip.l2c_repl` so that the L2 replacement policy treats lines brought in by SPP differently from demand lines. The change is guarded by a compile-time flag `TASK_3`. With `TASK_3` set to 0 the file reproduces the task 2b, so both can be built from the same source.

Every cache block already has a `prefetch` bit, which is set when a prefetch fills the line and cleared on its first demand hit, and the L2C already keeps `pf_fill` (prefetched lines filled into L2) and `pf_useful` (prefetched lines later used by a demand). Since the replacement functions are members of the cache, the L2 policy can read these directly.

A prefetch fill (`type == PREFETCH`) is inserted at RRPV = 3, the most distant position. At L2 only a small fraction of prefetched lines are later used by a demand because it is further away than L1. These lines are only coming to L2 if the MSHRs at L1 are occupied or if L1 evicts the line, so it's a low chance that this kind of data will be useful again. In case L2 prefetches turn out to be useful (`pf_fill` > 0 and more than 25% of prefetched lines used, i.e. $4 \cdot$ `pf_useful` $>$ `pf_fill`), they are inserted at RRPV = 2 instead, the same as data.

A prefetch request that hits in L2 does not change the line's RRPV. A prefetch touching a line says nothing about whether the program will re-use it, and promoting on it would make prefetched lines look hot. The first demand hit on a prefetched line shows that the prefetch was useful.

When choosing a victim at RRPV = 3, the policy first looks for a line that was prefetched and has not yet been used by a demand (its `prefetch` bit is still set), then for a data line (only when the follower set is in I-RRIP mode), and then for any line. If no line is at RRPV = 3, the set is aged and the search repeats. The set-dueling counters count only demand `LOAD`s, so prefetch traffic does not change the instruction ratio that decides between I-RRIP and RRIP.

6.2 Results

The results are split into two tables: L1D-side metrics and overall performance first, then L2-side MPKI.

Table 5: Task 3 results: IPC and L1D metrics. Speedup is relative to the LRU baseline (no prefetching). "Dyn I-RRIP" is Task 2B with no prefetcher; "Task 3" is Dyn I-RRIP with `TASK_3` set to 1.

Trace	Configuration	IPC	Speedup	L1D acc. (%)	L1D cov	L1D load MPKI
Trace1	SPP + LRU	0.233011	1.0029	52.8476	0.0267	10.3921
Trace1	Dyn I-RRIP	0.232271	0.9996	nan	nan	10.679
Trace1	Task 3	0.232945	1.0026	52.8775	0.0266	10.3938
Trace2	SPP + LRU	0.251268	1.0004	46.1722	0.0082	13.4266
Trace2	Dyn I-RRIP	0.255444	1.0171	nan	nan	13.5399
Trace2	Task 3	0.255606	1.0177	46.5894	0.0082	13.4267
Trace3	SPP + LRU	0.382230	1.0004	20.0238	0.0074	3.37792
Trace3	Dyn I-RRIP	0.381757	0.9992	nan	nan	2.1252
Trace3	Task 3	0.382013	0.9999	20.1283	0.0075	3.37796

Table 6: Task 3 results: L2 load MPKI. Configurations as in Table 5.

Trace	Configuration	L2C load MPKI	L2 instr. MPKI	L2 data MPKI
Trace1	SPP + LRU	17.4297	10.3602	7.06956
Trace1	Dyn I-RRIP	17.5401	10.0528	7.48724
Trace1	Task 3	17.2674	10.0577	7.20964
Trace2	SPP + LRU	27.3804	18.9224	8.45796
Trace2	Dyn I-RRIP	23.2616	14.4482	8.81336
Trace2	Task 3	23.1782	14.435	8.74328
Trace3	SPP + LRU	3.90212	1.12788	2.77424
Trace3	Dyn I-RRIP	4.0358	1.17076	2.86504
Trace3	Task 3	3.97436	1.16684	2.80752

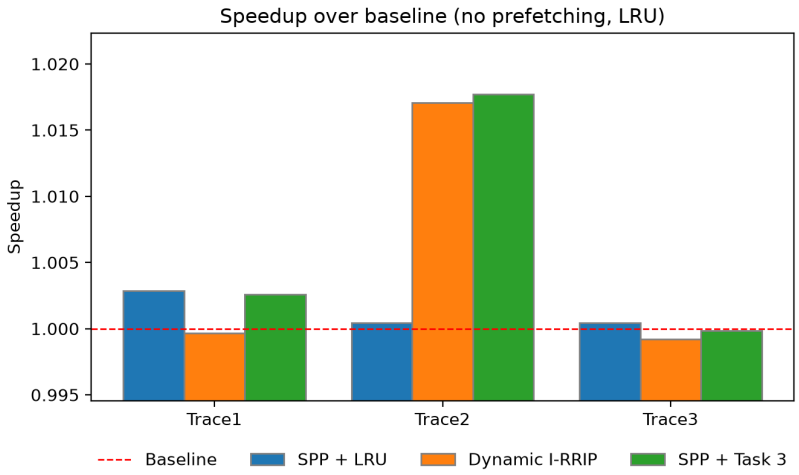


Figure 9: Task 3: Speedup of SPP+LRU, dynamic RRIP, and SPP+dynamic RRIP over the baseline.

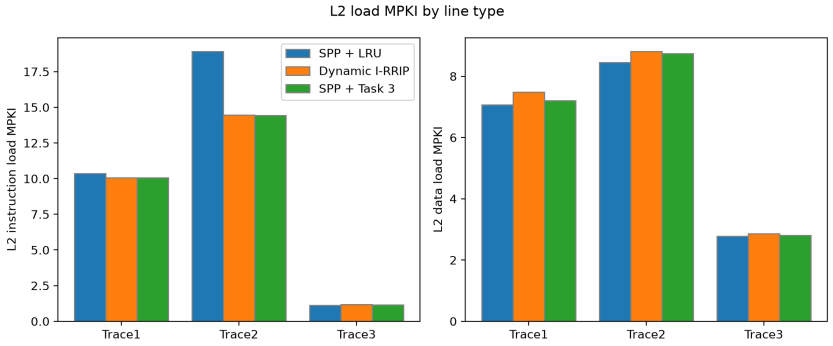


Figure 10: Task 3: L2 instruction and L2 data load MPKI.

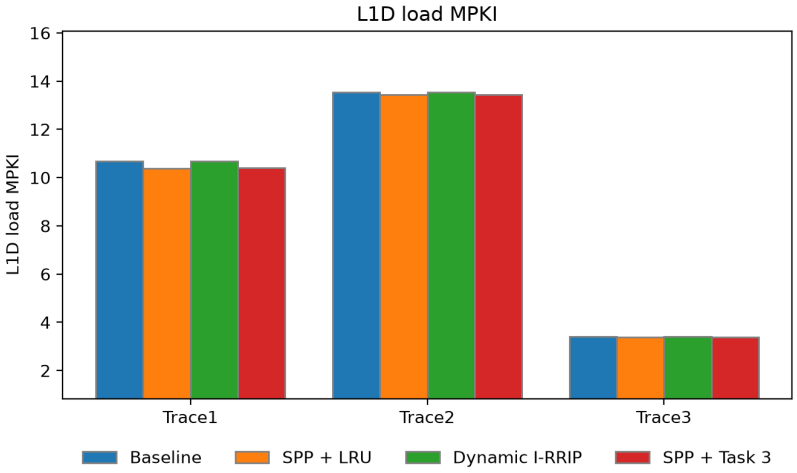


Figure 11: Task 3: L1D load MPKI.

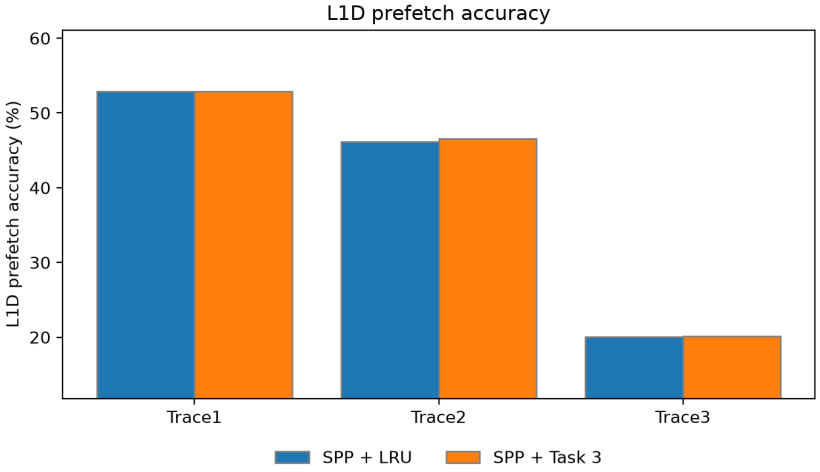


Figure 12: Task 3: L1D prefetch accuracy.

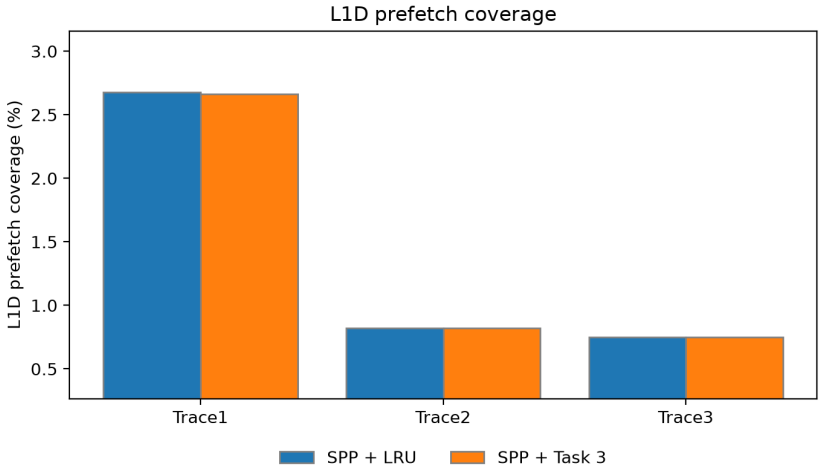


Figure 13: Task 3: L1D prefetch coverage.

6.3 Discussion

Adding SPP improves on dynamic I-RRIP on every trace. The prefetch accuracy of dynamic I-RRIP+SPP slightly increases, possibly because useful instructions are kept in the cache long enough to go to L1D, where they manage to contribute to the correct patterns in prefetching. This is especially seen in the instruction-heavy trace 2, which is also the only trace where IPC increased.

On Trace1 and Trace3, the IPC is slightly below SPP with LRU, partly because dynamic I-RRIP itself costs a little there, due to the limitations listed in task 2b.

Another reason is that the insertion policy for the prefetches may be too aggressive; we are inserting them at RRPV = 3, so many times they are kicked out of L2 before being used. This is supported by L2C prefetch accuracy dropping and LLC prefetch accuracy increasing from SPP to SPP+dynamic I-RRIP.